2. Foundation of SQA

What Is Software (SQA Perspective)?

Software includes all four of the following — not just code: Computer programs (code), Procedures, Documentation, Data

What Is Software Quality?

According to Pressman, software quality is the degree of conformance to:

- Specific functional requirements
- Specified software quality standards
- Good Software Engineering Practices (GSEP)

What Is Software Quality Assurance (SQA)?

According to the IEEE, SQA is a systematic and planned set of actions providing confidence that software development or maintenance conforms to:

- Established functional and technical requirements
- Managerial requirements such as schedule and budget

Main Objectives of SQA

- Assure conformance to functional and technical requirements
- Assure conformance to schedule and budget requirements
- Initiate and manage improvement of software development and SQA activities

Quality Assurance vs Quality Control

Aspect	Quality Assurance (QA)	Quality Control (QC)
Focus	Process-related	Product-related
When	Throughout development	After development / before shipping
Goal	Prevent errors, detect early	Inspect / test product
Software	→ SQA	→ Tester's role

Relationship Between Software Engineering and SQA

Software engineering provides a disciplined environment that supports SQA goals. Software engineers and the SQA team must cooperate to achieve quality efficiently and economically.

3. Why SQA in Software Is Different

Characteristic	Software Products	Other Industrial Products
Complexity	Very complex; huge number of operational options	Lower complexity; fewer operational options
Visibility	Invisible; defects cannot be detected by sight	Visible; many defects detectable by sight
Development	Defect detection mainly in one phase: development	Defect detection across development, planning, and manufacturing

Memory aid — Software is harder to assure because it is more complex, less visible, and more dependent on the development process itself.

4. Characteristics of SQA Environments

SQA must operate within a network of cooperating teams:

- Our software development team
- Other software development teams (same organization)
- Hardware development team
- Customer's development team
- Other suppliers' development teams

Key Environmental Pressures

- Contract conditions define scope and timetable
- Customer-supplier relationships require consultation and approval
- Teamwork requirements are strong
- Cooperation needed with internal and external software/hardware teams
- Interfaces with other software systems must be managed carefully
- Maintenance may continue for several years after development

5. Software Error, Fault and Failure

Term	Meaning	Origin
Error	A human mistake	Made by analyst, programmer, etc.
Fault	A software defect created by an error	Incorrect section of code / document
Failure	Incorrect behavior observed when a fault is activated	Occurs during execution

Memory chain: Error → Fault → Failure (Human action → Defect in software → Wrong behavior during execution)

Important Relationships

- Not every error becomes a fault
- Not every fault causes a visible failure immediately
- A fault may remain hidden until a certain condition triggers it
- If a faulty component is built into a larger system, the whole system may fail

Why Software Errors Occur

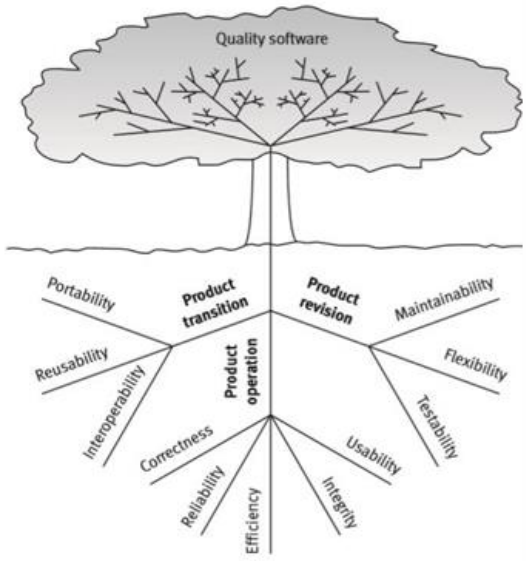
Common pressures that increase mistakes:

- Deadlines
- System complexity
- Organizational complexity
- Changing technology

Problems Caused by Faulty Software

- Loss of money or time
- Loss of business reputation
- Injury or death
- Aircraft crashes, hospital life-support failure, incorrect billing

McCall's factor model tree.



6. Nine Causes of Software Errors

1. **Faulty Requirements Definition**: Wrong, missing, incomplete, or unnecessary requirements.
2. **Client–Developer Communication Failures**: Misunderstanding written/oral changes, ignoring client messages.
3. **Deliberate Deviations from Requirements:** Reusing old modules without analysis, omitting functions due to time/budget, unapproved improvements.
4. **Logical Design Errors**: Wrong algorithms, sequencing errors, wrong boundary conditions, missing software states.
5. **Coding Errors**: Misunderstanding design docs, linguistic errors, wrong data selection.
6. **Non-compliance with Documentation/Coding Standards**: Makes code harder to understand, review, test, correct, and maintain.
7. **Shortcomings of the Testing Process:** Incomplete plans, untested functions, failure to document/correct detected faults.
8. **Procedure Errors**: Errors in procedures that guide users through complex multi-step processes.
9. **Documentation Errors**: Omissions, wrong instructions, listing non-existent or obsolete functions.

7. Five Views of Software Quality

View	Core Idea
Transcendental	Quality recognized through experience; difficult to define; product 'stands out'
User	Fitness for use; meets user needs; subjective (usability, reliability)
Manufacturing	Conformance to requirements; process-focused; build it right the first time
Product	Good internal properties (e.g. modularity) lead to good external quality
Value-based	Trade-off between cost and quality; what customers are willing to pay

8. McCall's Software Quality Factor Model

McCall's classic model contains 11 quality factors in 3 categories:

Category	Main Idea	Factors
Product Operation	Quality during day-to-day use	Correctness, Reliability, Efficiency, Integrity, Usability
Product Revision	Quality when maintaining the software	Maintainability, Flexibility, Testability
Product Transition	Quality when moving / integrating	Portability, Reusability, Interoperability

Memory: Operation = using now | Revision = changing later | Transition = moving elsewhere

Product Operation Factors

Factor	Focus	Key Sub-factors
Correctness	Required outputs	Accuracy, Completeness, Up-to-dateness, Availability
Reliability	Failure rate / recovery	System reliability, Application reliability, Failure recovery
Efficiency	Hardware resource usage	Processing, Storage, Communication, Power usage
Integrity	Security / access control	Access control, Access audit
Usability	Staff training & operation	Operability, Training

Product Revision Factors

Factor	Focus	Key Sub-factors
Maintainability	Finding & fixing failures	Simplicity, Modularity, Self-descriptiveness, Document accessibility
Flexibility	Adaptive maintenance	Modularity, Generality, Simplicity, Self-descriptiveness
Testability	Testing the system & ops	User testability, Failure maintenance testability, Traceability

Product Transition Factors

Factor	Focus	Key Sub-factors
Portability	Moving to different environments	SW system independence, Modularity, Self-descriptive
Reusability	Using modules in other projects	Modularity, Document accessibility, Application independence
Interoperability	Interfaces with other systems/firmware	Commonality, System compatibility, SW system independence

9. Testing Basics

What Is Testing?

Testing is an activity used to reduce risk and improve quality by finding defects. Important points:

- Testing does not directly remove defects or improve quality by itself
- Testing reports defects so the development team can fix them
- Testing also measures some aspects of software quality through systematic coverage

Static vs Dynamic Testing

Type	Definition	Examples
Static Testing	Testing without executing code	Reviews, checking specs and documents
Dynamic Testing	Testing by executing the code	Running tests with test data

Retesting vs Regression Testing

Type	Focus	Purpose
Retesting	The specific fix — narrow focus	Confirms the reported defect is corrected
Regression Testing	Side effects — broader focus	Protects existing functionality from unintended damage

10. Principles of Testing

1. Testing Shows the Presence of Bugs
Testing can show that defects exist, but cannot prove that no defects exist.

2. Exhaustive Testing Is Impossible
Focus on risk, priority, and important areas rather than testing every input and condition.

3. Early Testing
The earlier a defect is found, the cheaper it is to fix. Defects in requirements are most expensive when found late.

4. Defect Clustering
About 80% of problems are found in about 20% of modules (Pareto principle). Complexity, volatile code, and developer experience affect clustering.

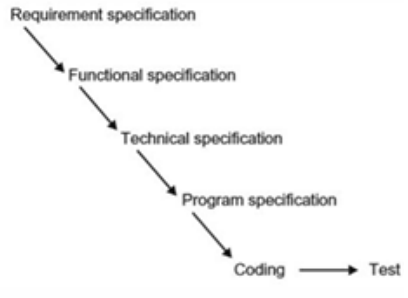
5. The Pesticide Paradox
Repeating the same tests eventually stops finding new defects. Test cases should be reviewed and refreshed regularly.

6. Testing Is Context Dependent

Different kinds of software need different kinds of testing.

7. Absence of Errors Fallacy

Even zero known defects does not mean the software is fit for release if it does not meet user needs.



11. SDLC Models

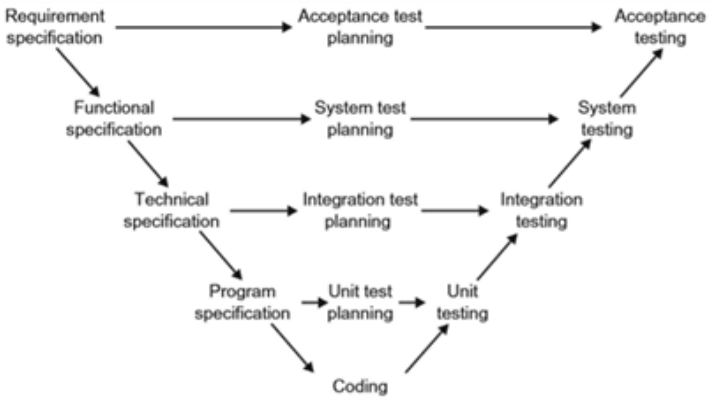
Waterfall Model

Aspect	Detail
Flow	Requirement Spec → Functional Spec → Technical Spec → Program Spec → Coding → Test
Strength	Clear sequential stages
Weakness	Testing mainly at the end; defects may be discovered too late

The V-Model

In the V-model, each development work-product has a matching test level. Test planning starts early, not after coding.

Development Work-Product (Left Side)	Test Level (Right Side)
Requirement Specification	Acceptance Testing
Functional Specification	System Testing
Technical Specification	Integration Testing
Program Specification	Unit Testing
Coding	Actual code under test



Benefit: Defects can be found earlier. Limit: User validation still happens late if requirements were wrong.

Verification vs Validation

Concept	Question	Example
Verification	Are we building the product the RIGHT WAY?	Does the website follow usability guidelines?
Validation	Are we building the RIGHT PRODUCT?	Can novice users actually use the website easily?

12. Test Levels

Level	Main Focus	Typical Tester	Test Bases
Unit (Component)	Single component / module	Developer	Component requirements, detailed design, code
Integration	Interfaces / interactions between units	Developer / Tester	System design, architecture, use cases
System	Full system end-to-end	Tester	System requirements, functional specs, risk analysis
Acceptance	User perspective / business expectations	Customer / User	User requirements, business processes

Unit Testing

- Checks that a single component meets its specification and executes all its code
- Requires access to the code; usually done by the developer who wrote it
- Tools: unit test frameworks, debugging tools, Test-Driven Development (TDD)
- Defects found during unit testing are often not formally recorded

Integration Testing

Exposes defects in interfaces and interactions between integrated components or systems.

Integration Strategies

Strategy	Description
Big Bang	All units linked at once to form the complete system
Top-Down	Start with higher-level components; stubs replace lower-level ones
Bottom-Up	Build from lower-level components upward; drivers simulate higher-level calls

Integration Levels

Level	Focus	Done By
Component Integration	Interactions between software components (after unit testing)	Developers
System Integration	Interactions between different systems (e.g. trading ↔ exchange)	Testers

System Testing

- Examines the full system from an end-to-end perspective
- Checks both functional and non-functional requirements
- Test bases: system/software requirement specs, use cases, risk analysis reports

Acceptance Testing

- Gives end users confidence that the system meets their expectations
- Based mainly on the requirement specification
- Forms: User acceptance, Operational acceptance, Contract/regulation, Alpha (developer site), Beta (customer site)

13. Quick Revision Comparisons

Error vs Fault vs Failure

Term	Meaning
Error	Human mistake
Fault	Defect created in software or documentation
Failure	Observable incorrect behavior during execution

Static vs Dynamic Testing

Type	Definition
Static Testing	No code execution — reviews, document checks
Dynamic Testing	Code is executed — running tests with data

Waterfall vs V-Model

Aspect	Waterfall	V-Model
Testing starts	After coding is complete	Conceptually at each spec stage
Defect discovery	Risk of late discovery	Earlier alignment
Structure	Simple sequential flow	Development ↔ test level mapping